

MIPS SUMMARY (ASSEMBLY)

Link tải phần mềm QTSpim: [Tại đây](#)

1. Program Template

The following is a very basic template for QtSpim MIPS programs. This general template will be used for all programs.

```
# Name and assignment number

# -----
# Data declarations go in this section.

.data
#   program specific data declarations

# -----
# Program code goes in this section.

.text
.globl    main
.ent      main
main:

# -----
#   your program code goes here.

# -----
# Done, terminate program.

        li    $v0, 10
        syscall                # all done!
.end main
```

2. Data Declarations

The data type must be preceded with a "." (period). The general format is:

<variableName>: .<dataType> <initialValue>

The supported data types are as follows:

Declaration	
.byte	8-bit variable(s)
.half	16-bit variable(s)
.word	32-bit variable(s)
.ascii	ASCII string
.asciiz	NULL terminated ASCII string
.float	32 bit IEEE floating point number
.double	64 bit IEEE floating point number
.space <n>	<n> bytes of uninitialized memory

Assuming the following data declarations:

```
num:      .word      0
wnum:     .word      42
hnum:     .half      73
bnum:     .byte      7
wans:     .word      0
hans:     .half      0
bans:     .byte      0
```

To perform, the basic operations of:

```
num = 27
wans = wnum
hans = hnum
bans = bnum
```

The following instructions could be used:

```
li    $t0, 27
sw    $t0, num           # num = 27
lw    $t0, wnum
sw    $t0, wans          # wans = wnum
lh    $t1, hnum
sh    $t1, hans          # hans = hnum
```

3. Load / Store Instructions

The general forms of the load and store instructions are as follows:

Instruction	Description
l<type> Rdest, mem	Load value from memory location memory into destination register.
li Rdest, imm	Load specified immediate value into destination register.
la Rdest, mem	Load address of memory location into destination register.
s<type> Rsrc, mem	Store contents of source register into memory location.

a) load: loading of data from memory (e.g., variables or arrays) into registers

- **syntax:**

*l <type> register_destination, RAM_source # Load integer
number*

*l. <type> register_destination, RAM_source # Load float
number*

b) load immediate:

- **syntax:**

li register_destination, value # Load immediate of a integer number

li.<type> register_destination, value # Load immediate of a float number

c) store: storing of data in register back to memory

- **syntax:** *s <type> register_source, RAM_destination # Store integer number*

s.<type> register_source, RAM_destination # Store float number

4. Integer Arithmetic Operations

Instruction		Description
add	Rdest, Rsrc, Src	Signed addition $Rdest = Rsrc + Src \text{ or } Imm$
addu	Rdest, Rsrc, Src	Unsigned addition $Rdest = Rsrc + Src \text{ or } Imm$
sub	Rdest, Rsrc, Src	Signed subtraction $Rdest = Rsrc - Src \text{ or } Imm$
subu	Rdest, Rsrc, Src	Unsigned subtraction $Rdest = Rsrc - Src \text{ or } Imm$
mul	Rdest, Rsrc, Src	Signed multiply with no overflow $Rdest = Rsrc * Src \text{ or } Imm$
mulo	Rdest, Rsrc, Src	Signed multiply with overflow $Rdest = Rsrc * Src \text{ or } Imm$
mulou	Rdest, Rsrc, Src	Unsigned multiply with overflow $Rdest = Rsrc * Src \text{ or } Imm$
mult	Rsrc1, Rsrc1	Signed 64-bit multiply \$hi/\$lo = $Rsrc1 * Rsrc2$

multu	Rsrc1, Rsrc1	Unsigned 64-bit multiply \$hi/\$lo = Rsrc1 * Rsrc2
div	Rdest, Rsrc, Src	Signed divide Rdest = Rsrc / Src or Imm
divu	Rdest, Rsrc, Src	Unsigned divide Rdest = Rsrc / Src or Imm
div	Rsrc1, Rsrc1	Signed divide with remainder \$lo = Rsrc1 / Rsrc2 \$hi = Rsrc1 % Rsrc2
divu	Rsrc1, Rsrc1	Unsigned divide with remainder \$lo = Rsrc1 / Rsrc2 \$hi = Rsrc1 % Rsrc2
rem	Rdest, Rsrc, Src	Signed remainder Rdest = Rsrc % Src or Imm
remu	Rdest, Rsrc, Src	Unsigned remainder Rdest = Rsrc % Src or Imm
abs	Rdest, Rsrc	Absolute value Rdest = Rsrc
neg	Rdest, Rsrc	Signed negation Rdest = - Rsrc
negu	Rdest, Rsrc	Unsigned negation Rdest = - Rsrc

Assuming the following data declarations:

```

wnum1:    .word    651
wnum2:    .word    42
wans1:    .word    0
wans2:    .word    0
wans3:    .word    0
hnum1:    .half    73
hnum2:    .half    15

```

```

hans:      .half      0
bnum1:     .byte      7
bnum2:     .byte      9
bans:      .byte      0

```

To perform, the basic operations of:

```

wans1 = wnum1 + wnum2
wans2 = wnum1 * wnum2
wans3 = wnum1 % wnum2
hans  = hnum1 * hnum2
bans  = bnum1 / bnum2

```

The following instructions could be used:

```

lw    $t0, wnum1
lw    $t1, wnum2
add   $t2, $t0, $t1
sw    $t2, wans1           # wans1 = wnum1 + wnum2

lw    $t0, wnum1
lw    $t1, wnum2
mul   $t2, $t0, $t1
sw    $t2, wans2           # wans2 = wnum1 * wnum2

lw    $t0, wnum1
lw    $t1, wnum2
rem   $t2, $t0, $t1
sw    $t2, wans3           # wans = wnum1 % wnum2

lh    $t0, hnum1
lh    $t1, hnum2

mul   $t2, $t0, $t1
sh    $t2, hans            # hans = hnum1 * hnum2

lb    $t0, bnum1
lb    $t1, bnum2
div   $t2, $t0, $t1
sb    $t2, bans            # bans = bnum1 / bnum2

```

In this case, the floating-point types are “.s” for single-precision and “.d” for double-precision.

Assuming the following data declarations:

```
fnum1:    .float      3.14
fnum2:    .float      0.0
dnum1:    .double     6.28
dnum2:    .double     0.0
```

To perform, the basic operations of:

```
fnum2 = fnum1
dnum2 = dnum1
```

The following instructions :

```
l.s      $f6, fnum1
s.s      $f6, fnum2                # fnum2 = fnum1

l.d      $f6, dnum1
mov.d    $f8, $f6                 # unnecessary use of mov
                                           # just as an example
s.d      $f8, dnum2                # dnum2 = dnum1
```

5. Convert số nguyên thành số thực (Xem chi tiết mục 5.7.3)

Giả sử để chia 2 số nguyên x, y và kết quả sau khi chia phải hiển thị là kiểu số thực thì chúng ta phải convert 2 số nguyên x, y thành số thực trước khi chia.

VD:

```
lw      $t1, 3      # y=2
lw      $t2, 2      # x=3

mtc1    $t1, $f1     # switch register $t1 to $f1
mtc1    $t2, $f2     # switch register $t2 to $f2
cvt.s.w $f1, $f1     # convert integer $f1 to float $f1
cvt.s.w $f2, $f2     # convert integer $f2 to float $f2
div.s   $f3, $f2, $f1 # Divide float $f3 = x/y
```

6. System Calls and I/O

Service	System call code	Arguments	Result
print_int	1	\$a0=integer	
print_float	2	\$f12=float	
print_double	3	\$f12=double	
print_string	4	\$a0=string	
read_int	5		integer (in \$v0)
read_float	6		float (in \$f0)
read_double	7		double (in \$f0)
read_string	8	\$a0=buffer, \$a1=length	
sbrk	9	\$a0=amount	
exit	10		

- Read a integer number**
li \$v0, 5 # load code 5 vào \$v0 để hệ thống nhận biết nhập số nguyên
syscall # Gọi hệ thống để nhập số nguyên và lưu số vừa nhập vào thanh ghi \$v0
- Read a float**
li \$v0, 6 # load code 6 vào \$v0 để hệ thống nhận biết nhập số float
syscall # Gọi hệ thống để nhập số float và lưu số vừa nhập vào thanh ghi \$f0
- Read a double**
li \$v0, 7 # load code 7 vào \$v0 để hệ thống nhận biết nhập số double
syscall # Gọi hệ thống để nhập số double và lưu số vừa nhập vào thanh ghi \$f0
- Print a integer**
li \$v0, 1 # load code 1 vào \$v0 để hệ thống nhận biết xuất số nguyên
lw \$a0, number # Load số cần xuất vào thanh ghi \$a0
syscall # Gọi hệ thống để xuất giá trị chưa trong thanh ghi \$a0
- Print a float**
li \$v0, 2 # load code 2 vào \$v0 để hệ thống nhận biết xuất số float
mov.s \$f12, \$f0 # Di chuyển giá trị trong \$f0 vào \$f12
syscall # Gọi hệ thống để xuất giá trị float chứa trong thanh ghi \$f12

- **Print a double**

```
li $v0, 3          # load code 3 vào $v0 để hệ thống nhận biết xuất số double
mov.s $f12, $f0    # Di chuyển giá trị trong $f0 vào $f12
syscall            # Gọi hệ thống để xuất giá trị double chứa trong thanh ghi $f12
```

- **Print a string**

```
li $v0, 4          # load code 4 vào $v0 để hệ thống nhận biết xuất chuỗi
la $a0, msg        # Load địa chỉ của chuỗi cần xuất vào thanh ghi $a0
syscall            # Gọi hệ thống để xuất giá trị chuỗi chứa trong thanh ghi $a0
```

- **Exit**

```
li $v0, 10         # load code 10 vào $v0 để hệ thống nhận biết thoát chương trình
syscall            # Gọi hệ thống để thoát
```

Ví dụ 1: Viết chương trình Xuất một chuỗi “Xin chào các bạn” ra màn hình

Mở Editor soạn thảo code, lưu lại với đuôi .s (VD mở Notepad++, soạn code và Save: Bai1.s)

```
.data
    input:      .asciiiz    "Xin chao cac ban"

.text
.globl main
main:
    # print input
    li $v0, 4          # Load code =4 vào thanh ghi $v0 để hệ thống biết print
    chuỗi
    la $a0, input      # Load địa chỉ của chuỗi input cần xuất vào $a0
    syscall            # Gọi hệ thống để xuất nội dung trong $a0 ra
    # exit
    li $v0,10         # Load code =10 vào thanh ghi $v0 để hệ thống biết là exit
    syscall            # Gọi hệ thống để exit
```

Ví dụ 2: Viết chương trình nhập vào 2 số nguyên. Tính tổng và xuất kết quả

Mở Editor soạn thảo code, lưu lại với đuôi .s (VD mở Notepad++, soạn code và Save: Bai1.s)

```
.data
    msg:      .asciiiz    "Nhap 2 so nguyen de tinh tong, cach nhau boi Enter: \n"

.text
.globl main
main:
    # Print string msg
    li $v0, 4
    la $a0, msg
```

```

        syscall

# Nhập số nguyên thứ nhất
        li $v0, 5          # Load code =5 vào thanh ghi $v0 để hệ thống biết là nhập số
nguyên
        syscall          # Gọi hệ thống để cho phép nhập số nguyên, và lưu số vừa nhập
vào $v0
        move $t1, $v0 # Di chuyển giá trị của thanh ghi $v0 vào $t1 ($t1 = số thứ 1)

#Nhập số nguyên thứ hai
        li $v0, 5          # Load code =5 vào thanh ghi $v0 để hệ thống biết là nhập số
nguyên
        syscall          # Gọi hệ thống để cho phép nhập số nguyên, và lưu số vừa nhập
vào $v0
        move $t2, $v0 # Di chuyển giá trị của thanh ghi $v0 vào $t2 ($t2=số thứ 2)

# Tính tổng
        add $t3, $t1, $t2    # $t3 = $t1+$t2

#print a integer
        li $v0, 1          # Load code=1 để hệ thống nhận biết print a integer
        move $a0, $t3 # Di chuyển giá trị trong $t3 vào $a0 để xuất ra
        syscall          # Gọi hệ thống để xuất giá trị trong $a0 ra ngoài
# Thoát
        li $v0, 10
        syscall

```

Ví dụ 3: Viết chương trình nhập vào 02 số thực. Tính tổng 02 số thực vừa nhập

```

.data
    msg: .asciiz "Nhập 2 số thực để tính tổng, cách nhau bởi Enter: \n"
.text
.globl main
main:
    # Print string msg
        li $v0, 4
        la $a0, msg
        syscall

    # Nhập số float thứ 1
        li $v0, 6          # Load code =6 vào thanh ghi $v0 để hệ thống biết là nhập
số float

```

```

        syscall                # Gọi hệ thống để cho phép nhập số float, và lưu số vừa
nhập vào $f0
        mov.s $f1, $f0        # Di chuyển giá trị của thanh ghi $f0 vào $f1 ($f1 = số thứ
1)

        #Nhao so float thu 2
        li $v0, 6             # Load code =6 vào thanh ghi $v0 để hệ thống biết là nhập số float
        syscall                # Gọi hệ thống để cho phép nhập số float, và lưu số vừa nhập vào
$f0
        mov.s $f2, $f0        # Di chuyển giá trị của thanh ghi $f0 vào $f2 ($f2=số thứ 2)

        # Tính tổng
        add.s $f3, $f1, $f2    # $f3 = $f1+$f2

        #print float
        li $v0, 2             # Load code=2 để hệ thống nhận biết print a float
        mov.s $f12, $f3        # Di chuyển giá trị trong $f3 vào $f12 để xuất ra
        syscall                # Gọi hệ thống để xuất giá trị trong $f12 ra ngoài
        # Thoát
        li $v0, 10
        syscall

```

Ví dụ 3: Viết chương trình tính diện tích, chu vi

Data Declarations

.data

```

pi: .float 3.14159
fourPtZero: .float 4.0
threePtZero: .float 3.0
radius: .float 17.25
surfaceArea: .float 0.0
volume: .float 0.0

```

.text

.globl main

main:

```

l.s    $f2, fourPtZero
l.s    $f4, pi
mul.s  $f4, $f2, $f4        # 4.0 * pi
l.s    $f6, radius          # radius
mul.s  $f8, $f6, $f6        # radius^2
mul.s  $f8, $f4, $f8        # 4.0 * pi * radius^2
s.s    $f8, surfaceArea     # store final answer

```

